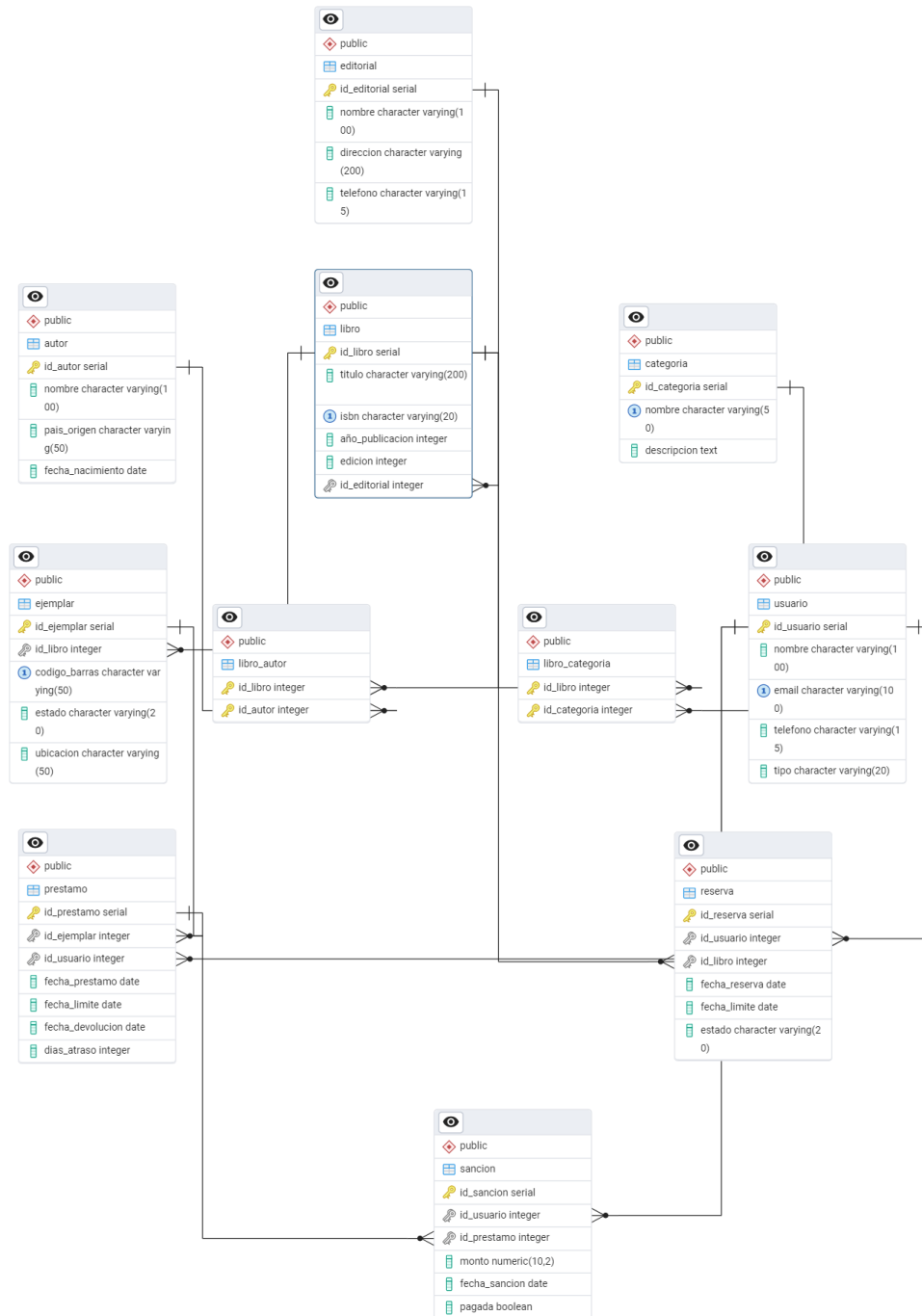


Proyecto #3 – Sistema de Gestión Editorial

Diagrama ER



Reflexión:

1. Criterios para la definición de entidades y relaciones

El diseño del modelo se basó en un análisis exhaustivo de los procesos operativos de una editorial típica. Se identificaron como entidades fundamentales los libros, ejemplares físicos, autores, usuarios, préstamos y reservas, dado que representan los elementos centrales del flujo de trabajo. Para las relaciones, se priorizó la fidelidad al modelo de negocio real: se estableció una relación muchos-a-muchos entre libros y autores mediante una tabla puente (Libro_Autor), ya que un libro puede tener múltiples autores y un autor puede escribir múltiples libros. Las decisiones de simplificación incluyeron la consolidación de tipos de usuarios en un campo enumerado en lugar de una tabla separada, considerando que los privilegios de acceso eran similares para todos los roles. Se optó por no modelar el historial completo de cambios de estado para simplificar las consultas frecuentes, aunque esto implicó perder capacidad de auditoría detallada. La ubicación física de los ejemplares se representó como texto libre en lugar de un modelo jerárquico de estanterías, decisión que aunque limita ciertas búsquedas, simplificó considerablemente el esquema.

2. Adecuación de claves primarias y foráneas

La estrategia de claves implementada combinó distintos enfoques según la naturaleza de cada entidad. Para tablas maestras como Editoriales y Autores se utilizaron claves artificiales autoincrementales (SERIAL), que demostraron ser eficientes tanto en rendimiento como en facilidad de administración. En contraste, para la tabla Ejemplares se adoptó un código de barras como clave natural, aprovechando que este identificador ya existía en el dominio del problema y garantizaba unicidad. Todas las relaciones se implementaron con claves foráneas y acciones RESTRICT ON DELETE para preservar la integridad referencial. Durante el desarrollo, este diseño demostró ser particularmente efectivo en consultas que involucraban múltiples JOINS, como los reportes de préstamos por editorial, donde los índices implícitos en las claves primarias optimizaron significativamente el rendimiento. La única excepción notable fue en las consultas de reservas pendientes, donde la ausencia de un índice adicional en el campo 'estado' generó inicialmente cuellos de botella que luego fueron resueltos.

3. Aplicación de la normalización

El proceso de normalización se aplicó rigurosamente hasta la tercera forma normal (3FN). En la 1FN, se aseguró la atomicidad de los campos, eliminando atributos multivaluados como las categorías de libros mediante la creación de la tabla Libro_Categoría. Para alcanzar la 2FN, se eliminaron dependencias parciales extrayendo información de editoriales a una tabla separada, evitando así la redundancia de almacenar repetidamente datos como la dirección de la editorial. La 3FN se logró removiendo dependencias transitivas, como separar claramente la información de autores de la de sus libros. Sin embargo, se tomaron decisiones conscientes de desnormalización controlada: el campo calculado 'días_atraso' en la tabla Préstamos viola teóricamente la 3FN pero se justificó por el alto costo computacional de calcularlo en cada consulta. Otra excepción fue el manejo del estado de los ejemplares como un ENUM en lugar de una tabla relacionada, sacrificando flexibilidad futura a cambio de simplificar consultas frecuentes. Estas concesiones prácticas demostraron ser acertadas durante las pruebas de rendimiento.

4. Restricciones y reglas de negocio implementadas

El esquema incorporó diversas restricciones para garantizar la calidad de los datos. A nivel de columnas, se utilizaron NOT NULL para campos obligatorios como títulos de libros y fechas de operación, CHECK para validar rangos (ej: días_atraso ≥ 0) y UNIQUE para campos como ISBN y códigos de ejemplares. Las reglas de negocio más complejas se implementaron mediante triggers: el trigger 'calcular_fecha_limite' automatiza el cálculo de fechas de devolución (15 días después del préstamo), mientras que 'generar_sancion' aplica multas automáticas de \$5 por día de atraso. Particular atención merece el trigger 'completar_reserva', que gestiona la transición de estado cuando un ejemplar se devuelve, demostrando cómo la lógica de negocio crítica puede encapsularse eficientemente en la capa de base de datos. Estas implementaciones se justificaron por tres razones principales: (1) garantizar consistencia independientemente de qué aplicación acceda a los datos, (2) mejorar el rendimiento al evitar viajes adicionales a la capa de aplicación, y (3) centralizar el mantenimiento de reglas complejas.

5. Ventajas y desventajas en consultas complejas

El modelo demostró excelente desempeño en consultas típicas como la búsqueda de préstamos activos o el listado de libros por categoría, donde las relaciones bien definidas y los índices implícitos en claves primarias/foráneas permitieron ejecuciones eficientes. Sin embargo, se identificaron desafíos en escenarios más complejos: las consultas que combinaban préstamos, reservas y sanciones en un solo reporte requerían múltiples LEFT JOIN que impactaban el rendimiento cuando la base de datos crecía. Las búsquedas por rangos de fechas combinadas con filtros por categoría mostraron la necesidad de índices adicionales.

Un caso particularmente interesante fue el reporte de libros más populares, donde la agregación (COUNT) sobre tablas grandes benefició significativamente de la creación de un índice específico sobre fecha_prestamo. La flexibilidad del modelo se vio limitada principalmente en consultas que requerían acceso histórico a cambios de estado, una funcionalidad que quedó fuera del alcance inicial pero que sería prioritaria en una futura versión.

6. Mejoras para un entorno de producción

Para escalar el sistema a un entorno productivo, se requerirían modificaciones sustanciales en tres áreas principales: estructura, rendimiento y disponibilidad. En cuanto a estructura, se implementaría particionamiento horizontal en la tabla Préstamos por rangos de fecha, permitiendo distribuir la carga en los reportes históricos. Para el rendimiento, se crearían índices especializados en campos de filtrado frecuente (como estado de ejemplares y fechas) y vistas materializadas para reportes recurrentes. La disponibilidad mejoraría mediante réplicas de lectura para descargar las consultas analíticas del servidor principal. Adicionalmente, se implementaría un sistema de colas para procesar asíncronamente operaciones como la generación de reportes masivos. En el aspecto funcional, se añadirían tablas de historial para auditoría, triggers más robustos con manejo de concurrencia, y una API REST para integración con otros sistemas. Estas mejoras, aunque incrementarían la complejidad del sistema, mantendrían la claridad del diseño relacional mientras atienden los requerimientos de escalabilidad y robustez de un entorno productivo real.